

Acting in Parallel Without Talking: Focal Points, Collisions, and Length Signals in Diffusion-LM Agents and LLM Agent Teams

Anonymous Author(s)

Submission Id: 1241

“Can you check my loan eligibility for a home loan of amount \$500,000 from HSBC with annual income \$100,000 and for Wells Fargo for a amount of \$700,000 with annual income of \$120,000?”

Slot lengths	Call 1: institution, amount, income	Call 2: institution, amount, income	BFCL verdict	What went wrong
exact	HSBC, 500000, 100000	Wells Fargo, 700000, 120000	correct	—
one extra mask	HSBC Bank , 5000000, 1000000	Wells Fargo Bank , 7000000, 1200000	wrong	every value is lengthened; amounts ×10
swapped	Wells Fargo , 500000, 100000	HSBC , 700000, 120000	wrong	the banks are bound to the wrong calls

Figure 1: A dLLM agent’s parallel turn under three slot lengths. Dream fills the value slots of a fixed call skeleton committing one token per step, so no two decisions are ever made at once. With exact slot lengths the turn is correct. With one extra mask per slot, every value grows to fill its slot. With the two institutions’ slot lengths swapped, each call takes the bank whose name fits its slot: an error that looks like a coordination failure of parallel decoding, made without any parallel decoding.

Abstract

When an agent issues several tool calls at once, or a team of LLM agents splits a task, each decision must take its own role without seeing the others. We study where such roles come from, using diffusion language models (dLLMs) as a controllable test bed: a dLLM agent fills the value slots of a fixed call skeleton and can commit any number of slots in the same step, so its decoding schedule sets who moves together and who sees what. We compare its slots with teams of LLM agents on the same requests. On the parallel calls of the Berkeley Function Calling Leaderboard, slots that move together coordinate on the order in which the request mentions its entities: committing 16 tokens per step instead of one costs Dream-7B 5 points of set accuracy. Agents that act at the same time cannot use this order, even when numbered (28.8% set accuracy against 81.7% for same-step slots) or told the convention (48.1%); agents that take turns come within 4 points of the single agent. When nothing orders the choices, simultaneous slots and agents collide, sampling does not help, and turn-taking does, as does a confidence threshold that commits uncertain choices one at a time. The strongest cue is the slot length: a decider reads the number of masks as the length of its value, which assigns roles when the lengths are right and swaps them when they are wrong, even one token per step and for a single agent. A closing delimiter written after each slot doubles the share of values that one extra mask corrupts, but half of the loss remains when the model writes the delimiter itself. Errors from wrong lengths look like failures of parallel decoding, so evaluations of such agents must control slot lengths before blaming parallelism.

Keywords

LLM agents, multi-agent LLM systems, tool use, parallel action selection, role assignment, coordination without communication, focal points, diffusion language models

ACM Reference Format:

Anonymous Author(s). 2027. Acting in Parallel Without Talking: Focal Points, Collisions, and Length Signals in Diffusion-LM Agents and LLM Agent Teams. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027, IFAAMAS*, 9 pages.

1 Introduction

Agents increasingly act in parallel. A tool-calling agent answers one request with several calls at once, such as the weather in two cities or a loan check at two banks, and in multi-agent LLM systems several workers claim subtasks of a shared plan at the same time [19, 27]. Each parallel decision must then take its own role, such as an entity or a subtask, without seeing what the others take. If two decisions take the same role, an action is duplicated; if they swap roles, actions are bound to the wrong objects. Coordination without communication has well-known answers: a focal point that everyone notices [29], a shared convention [14], labels that break the symmetry, or taking turns. We ask which of them work for LLM-based deciders.

Diffusion language models (dLLMs) make this question precise. A dLLM agent can write all the calls of a turn on one canvas and commit many tokens in each denoising step. Tokens committed in the same step are chosen without seeing each other, so the decoding schedule decides which slots move together and what each one sees, and we can vary it exactly. dLLM agents also matter in their own right: they fail at tool calling, and the failures have been blamed on diffusion noise and parallel decoding [18]. To keep every call well formed, such an agent can write the syntax of its turn in advance as a skeleton and leave only the argument values masked. Each value then gets a slot with a fixed number of masks, set before the value is known. Figure 1 shows what this does even when the



This work is licensed under a Creative Commons Attribution International 4.0 License.

agent commits one token per step: with one extra mask per slot, HSBC becomes “HSBC Bank” and a \$500,000 loan becomes one of \$5,000,000; with the slot lengths of the two banks swapped, the banks change calls.

We treat the slots of such a turn as deciders that share one policy and differ only in their position and in what they observe, and we derive predictions from three cues they share: the order in which the request mentions its entities (a focal point), each decider’s role (a position on the shared skeleton, or a number in an agent’s prompt), and the slot length (a signal). We test them on the parallel calls of the Berkeley Function Calling Leaderboard [24] and on symmetric requests, with two dLLMs and an AR model, and we give the same requests to teams of LLM agents that each make one call, at the same time or in turns, anonymous, numbered, or told the convention.

We find three things. (1) When the request orders its entities, slots that move together coordinate on that order: committing 16 tokens per step instead of one costs Dream only 5.3 points of set accuracy. Agents that act at the same time cannot use the order: numbered Dream agents reach 28.8% set accuracy, and 48.1% when told the convention, against 81.7% for the same-step slots, while agents that take turns come within 4 points of the single agent. A role given as a position on a shared skeleton coordinates; a role given in a prompt does not. (2) When nothing orders the choices, simultaneous slots and agents collide, sampling does not help, and turn-taking prevents the collisions, as does a confidence threshold that commits uncertain choices one at a time. (3) The strongest cue is the slot length. A decider reads the number of masks as the length of its value: right lengths let anonymous agents that cannot see each other split the calls, and wrong lengths bind values to the wrong calls, even one token per step and for a single agent. A closing delimiter written after each slot doubles the share of values that one extra mask corrupts, but half of the loss remains when the model writes the delimiter itself. With slot lengths that the model estimates itself, one token per step produces about four times as many cross-call errors as 16 tokens per step with exact lengths, errors of the kind usually blamed on parallel decoding.

Our contributions are (i) a test bed and a model that separate the protocol, the role cues, and the action interface of parallel deciders (Section 3); (ii) evidence, across ordered and symmetric requests and across one agent and teams of agents, that roles anchored in shared structure coordinate parallel deciders, roles stated in prompts do not, and turn-taking always does (Sections 4 and 5); (iii) evidence that slot length acts as a signal that single deciders obey, with a control for the closing delimiter (Section 6); and (iv) an attribution protocol and design guidance for dLLM agents (Sections 7 and 8). Length biases of dLLMs are known for single spans [9, 17, 32]; we show what they do to the joint actions of agents.

2 Related Work

Tool-calling agents and parallel calls. LLM agents interleave reasoning with tool calls [30, 35], and many requests need several calls in one turn. LLMCompiler [13] plans such calls and executes them in parallel, and BFCL [24] evaluates parallel calls in dedicated categories. These works parallelize the execution of the calls; we study how one model generates them in parallel.

dLLM agents. Masked diffusion language models [5, 23, 28, 36] are being tried as agent backbones. Lu et al. [18] find that dLLM agents fail at tool calling, often with malformed JSON or wrong parameters, and attribute this to diffusion noise and to parallel decoding that weakens causal dependencies; they do not vary slot lengths or the number of tokens committed per step. With matched fine-tuning, dLLM agents run over 30% faster than AR agents but are more prone to structured tool-call failures [38]. DLLM-Searcher [37] has a dLLM search agent decode its tool call first and keep reasoning while the tool runs, and Qiu et al. [25] trace the retry loops of dLLM agents to the sampler committing its most confident positions first. Agents of Diffusion [12] uses two AR LLM agents to steer a frozen dLLM toward schema-conformant data. We control the agent’s action interface and decoding protocol separately.

Coordination without communication. Players who cannot talk coordinate on focal points [21, 29] and conventions [14]; in dispersion games they must instead spread over the actions [8]; zero-shot coordination avoids arbitrary symmetry-breaking conventions that independently trained partners would not share [10]. LLMs often find focal points without communicating [2], coordinate well when the environment decides but less when they must model their partners [1], and choose alike by default [3]. Parallel LLM workers coordinate through a shared attention cache [27] or a shared context and task queue that cuts redundant work [19]. We apply these notions to the deciders inside one agent’s decoding, whose shared policy makes any label-based convention available, and compare them with teams of LLM agents.

Parallel decoding and length in dLLMs. Tokens committed in the same step are sampled without their mutual dependencies [16]. Samplers bound the resulting error with a confidence threshold [31] or an entropy bound [4], and ParallelBench [11] documents invalid combinations such as “New City”. A fixed canvas length is a known limitation: code infilling degrades when the number of masks does not match the completion [32], fixed format anchors impose rigid spans that truncate content or add redundant content [9], and ample masks for schema fields lead to over-generation [33]. The remedies size a single span or response: CAL [17] searches for a span’s length, DreamOn [32] lets the model grow or shrink its canvas, DIA [9] estimates where a span ends, and ρ -EOS [34] and DAEDAL [15] adjust a response from end-of-sequence signals. Constrained decoding for dLLMs enforces grammars [22] or finite automata [6], as it does for AR models [7]; our skeleton is the simplest such constraint. We do not claim to discover the length bias. We measure it in actions with several slots, show that it binds values to the wrong calls, and show that these errors pass for coordination failures of parallel decoding.

3 Parallel Deciders, Protocols, and Role Cues

3.1 The Agent and Its Action Interface

A tool-calling agent receives a request q and a set of tool schemas $\mathcal{F}$, and acts by issuing a set of calls $A = \{a_1, \dots, a_n\}$, where each call $a_i = (f_i, \theta_i)$ names a tool $f_i \in \mathcal{F}$ and binds its arguments θ_i . When $n > 1$, the turn is a joint action. We score it as a set, as BFCL does [24]: the turn is correct if its calls can be matched one to one with the reference calls, whatever their order.

The dLLM agent writes A as a JSON array onto a canvas of mask tokens and fills the canvas over several denoising steps. To make every action well formed, its interface fixes the syntax of the turn in advance with a *skeleton*: the number of calls, the tool names, the argument names, and all brackets and quotes are written onto the canvas, and only the argument values are masked. Each value occupies a *slot* of ℓ mask tokens. The slots that hold the same argument in different calls of the same tool form a *sibling group*. Our skeleton is an oracle, built from the reference calls, so that every remaining error is an error in a value; Section 9 discusses what the oracle hides.

The skeleton writes the closing delimiter of each value after its slot: the closing quote of a string, or the comma or brace that follows any other value. A value ends early when the model writes its own closing token inside the slot; the rest of the slot is then set to padding (the model’s end-of-sequence token), and both are dropped from the output. Section 6 also tests a variant in which the slot holds the closing token itself and is padded with spaces. The autoregressive (AR) reference agent uses the same skeleton: it writes each value until its closing token and is cut off at the slot length.

3.2 Deciders and Protocols

At each denoising step, the dLLM predicts every masked position from the current canvas and commits some of them. A position committed in a step is chosen from its own predicted distribution given the canvas *before* that step, so positions committed in the same step cannot see each other’s choices. We call the masked positions *slot-level deciders*. They are positions of one network without preferences or beliefs of their own: they share one policy and differ only in their position on the canvas and in what has been committed when they move. A parallel turn is thus a common-payoff team decision problem [20] with a shared policy. We use the terms of game theory for its information structure, who moves when and who sees what, not for any reasoning by the deciders.

To compare, we give the same requests to *teams of n LLM agents*, one per call. Each agent is a separate generation of the same model, sees a skeleton of its own call only, and receives the request followed by a note on the protocol; the supplementary material gives the texts. A *protocol* says which deciders move together and what each sees:

- *Canvas*. With k tokens per step in confidence order, the k most confident masked positions move together. With $k = 1$, one decider moves at a time and sees every earlier commitment; we call this *one token per step*. Left-to-right decoding (LTR) fixes the order of moves. A confidence threshold τ [31] lets every position whose top probability exceeds τ move at once, and the most confident position alone when none does. LLaDA2.0 decodes in blocks of 32 tokens, causal across blocks; only 3.1% (parallel) and 2.3% (parallel_multiple) of the pairs of sibling slots share a block, so its calls almost always take turns.
- *Teams*. Agents act at the same time, seeing no other call, or in turns, seeing the calls made so far. They are anonymous (“You are one of n assistants”), numbered (“You are assistant i of n ”), or numbered and told the rule (“By convention,

assistant i makes the i -th of the calls, in the order in which the request mentions them”).

3.3 Role Cues and Predictions

The sibling deciders of a turn must take distinct entities, each the one its call needs. A shared policy can set them apart only through something they share that orders the candidates, or something that differs between them. We consider three such cues:

- (1) *Mention order*. The request names its entities in some order. Taking the i -th mentioned entity for the i -th call is a convention that any decider who knows its index can follow, and on set-scored requests it is a focal point among equally correct assignments [29].
- (2) *Role*. A slot’s position on the shared skeleton, or an agent’s number in its prompt. A role sets deciders apart only if the policy’s output depends on it, and helps only if something orders the candidates: “you are the second call” does not say which of all US cities is the second.
- (3) *Slot length*. Sibling values usually differ in length. In training, every mask hides exactly one token, so the mask count is a truthful signal of a value’s length in the sense of Lewis [14]; an interface that sets slot lengths sends this signal whether or not it is true.

These cues give three predictions, which Sections 4–6 test.

- P1** *Order coordinates*. When the request orders its entities, same-step slots take the i -th entity for the i -th call at any k , so committing many tokens per step costs little, even when slot lengths cannot tell the calls apart.
- P2** *Symmetric roles collide*. When nothing orders the candidates, deciders that move at the same time take the same value, and deciders that take turns do not.
- P3** *Length is a signal*. A decider reads its mask count as the length of its value: right lengths set siblings apart, and wrong lengths give a slot a longer version of its own value or a sibling’s value that fits, at any k , including one token per step and a single agent.

If a number in a prompt acts as a role, **P1** and **P2** also hold for numbered agents, which would take the i -th entity of an ordered request or of a list. Sections 4 and 5 test this.

3.4 Models, Data, and Measures

Models. Dream [36] (Dream-v0-Instruct-7B) is a dLLM with full bidirectional attention, initialized from Qwen2.5-7B. LLaDA2.0 [5] (LLaDA2.0-mini, a mixture of experts with 16B parameters, 1.4B of them active per token) is a block-diffusion dLLM that supports tool calling. The AR reference is Qwen2.5-7B-Instruct [26], the instruction-tuned version of the base model that Dream is initialized from. All models see the same system prompt, which lists the tools as JSON and asks for a JSON array of calls. All decode greedily, except the runs at $T = 0.7$ in Section 5.

Data. We use the parallel and parallel_multiple categories of BFCL v4 [24]: 400 requests with 2 to 8 calls (2.9 on average) and 3,063 value slots. LLaDA2.0’s runs with wrong slot lengths use 100 of them, 50 evenly spaced in each category. In the *equal-length* requests (213 for Dream’s tokenizer, 216 for LLaDA2.0’s), each

argument has slots of one length in every call that uses it. In the 104 *team-symmetric* requests, the sibling calls of each tool have the same arguments and slot lengths, so neither the skeleton nor the lengths tell apart the agents that call the same tool. For requests in which nothing tells the calls apart, we add 105 *choose-N* requests (Section 5).

Slot lengths. Exact: each slot has as many masks as its own reference value has tokens, tokenized on its own (the first acceptable value where BFCL lists several). *Surplus +s:* every slot gets $s \in \{1, 2, 4, 8\}$ extra masks (LLaDA2.0: no $s = 4$, and $k = 16$ only at $s = 1$). *Swap:* slot lengths are rotated by one position within each sibling group, on the requests where this changes a slot (165, and 34 for LLaDA2.0). *One slot longer:* in each sibling group with unequal lengths, one slot gets the length of the longest sibling value, and all others stay exact. These four derive from the reference values: they are diagnostic interventions, not lengths an agent could set. *Estimate:* lengths predicted by the model itself in one forward pass (Section 8).

Measures. *Set accuracy* is the BFCL verdict. The *cross-call error rate* (CCER) is the share of requests with at least one cross-call error, found by an optimal one-to-one matching of predicted and reference calls: a duplicated or missing call, a value that belongs to another call (cross-binding), a string spliced from two calls’ values (chimera), or an argument that the reference shares across calls but the prediction does not. At the slot level, we read each slot’s value from its own tokens and classify it as the slot’s own reference value, a sibling’s value (*fit* if it has exactly the slot’s length), an *overflow* (the own value followed by more), a truncation of the own value, or anything else. For paired comparisons on the same requests we give the difference with a 95% bootstrap interval over requests and an exact McNemar test, and for single rates a Wilson interval; tests are not corrected for multiplicity.

4 Ordered Requests: Mention Order Coordinates Slots, Not Agents

We first test **P1** on the BFCL requests with exact slot lengths (Table 1).

Same-step slots coordinate. Committing 16 tokens per step instead of one lowers Dream’s set accuracy from 89.8% to 84.5% (by 5.3 points, 95% interval [3.3, 7.5]) and LLaDA2.0’s from 87.8% to 79.8%, while cutting the forward passes per request by factors of 13 and 7. The confidence threshold keeps Dream at 89.0% with 1.9 forward passes per request; the 0.8-point loss is not significant ($p = 0.45$). Cross-call errors stay rare: Dream’s rate does not change significantly between $k = 1$ and $k = 16$ (1.3% and 2.0%, $p = 0.38$), LLaDA2.0’s rises from 2.5% to 4.5% ($p = 0.02$), and the AR agent’s is 1.3%. On the equal-length requests, where slot lengths cannot tell the calls apart, Dream’s set accuracy falls from 86.9% to 81.7%, much as on all requests. On those in the *parallel* category, the i -th call takes the i -th mentioned entity in 94.0% of Dream’s requests with one token per step and in 88.1% with 16; among the outputs that parse with every call matched, the share stays between 95.0% and 96.1% at every k . The cost of parallelism lies inside single values: of the 21 requests that Dream gets right with one token per step and wrong with 16, 13 have only single-call errors and 5 no longer

Table 1: One canvas, ordered requests: committing more tokens per step with exact slot lengths (BFCL *parallel* and *parallel_multiple*, 400 requests). Set accuracy and CCER in %; (eq.): on the requests in which each argument has slots of one length in every call that uses it (213 for Dream, 216 for LLaDA2.0), so that lengths cannot tell the calls apart; order: share of these requests in the *parallel* category (84 and 87) whose i -th call takes the i -th mentioned entity, counting outputs that do not parse as out of order; fwd: forward passes per request. LTR: left to right, one token per step; τ : commit every position above confidence 0.9. Qwen2.5 (autoregressive) in the same skeleton: 86.5% set accuracy, 1.3% CCER.

	$k=1$	2	4	8	16	LTR	τ
<i>Dream (full attention)</i>							
Set acc.	89.8	88.0	87.0	86.3	84.5	89.3	89.0
CCER	1.3	1.3	1.5	1.8	2.0	1.0	1.3
Set acc. (eq.)	86.9	85.4	84.0	82.6	81.7	86.4	85.4
Order	94.0	94.0	90.5	88.1	88.1	92.9	91.7
Fwd	28.0	14.2	7.3	3.9	2.2	27.8	1.9
<i>LLaDA2.0 (32-token blocks)</i>							
Set acc.	87.8	87.5	86.5	83.3	79.8	–	–
CCER	2.5	2.8	3.3	4.0	4.5	–	–
Set acc. (eq.)	87.5	88.0	86.1	83.8	80.6	–	–
Order	89.7	90.8	89.7	88.5	86.2	–	–
Fwd	27.8	14.8	8.3	5.1	3.9	–	–

parse, against 3 with cross-call errors (LLaDA2.0: 19, 7, and 7 of 33). BFCL usually lists the reference calls in mention order and our skeleton lays them out in that order, so following the order is also correct; and most values are copied from the request with high confidence, so the slots rarely face a real choice.

Simultaneous agents do not. On the 104 team-symmetric requests, the agents that call the same tool see the same skeleton, so only their prompts can set them apart (Table 2, left column). Anonymous agents that act at the same time receive identical inputs and make identical calls, so no team is correct. Numbering them helps Dream’s agents a little and Qwen2.5’s not at all: 28.8% and 0% set accuracy, against 81.7% for Dream’s canvas at 16 tokens per step on the same requests (difference 52.9 points, [42.3, 62.5]). Even when told the convention, they reach only 48.1% and 34.6%. Most numbered agents take the entity mentioned first (67.8% of Dream’s and 88.3% of Qwen2.5’s agents in sibling groups). Agents that take turns come close to the single agent but stay below it: Dream’s teams reach 84.6–85.6% against 88.5% for its canvas with one token per step, and Qwen2.5’s 88.5% against 91.3% for its single agent; each team loses 3 or 4 requests that the single agent gets right and gains at most one ($p \geq 0.125$).

P1 holds for same-step slots: a position on the shared skeleton acts as a role, and the request’s order assigns the entities to roles. It fails for agents that act at the same time, for whom a number in the prompt, and even a stated rule, is a weak role. Taking turns comes within 4 points of the single agent.

Table 2: One canvas against teams of agents. Ordered: set accuracy (%) on the 104 BFCL requests whose sibling calls share one single-call skeleton. Symmetric (choose-N): Dup. is the share of requests (%) in which two calls take the same city; Order, the share of list requests whose i -th call takes the i -th listed city. Agents act at the same time or in turns (seeing the calls made so far) and are anonymous, numbered (“assistant i of n ”), or numbered and told that assistant i makes the i -th call. Greedy decoding on 60 list and 45 open requests; $T=0.7$: five seeds. Open requests hold one-token slots, so only duplicates are reported.

	Ordered	Symmetric, greedy		$T=0.7$	
	Set	List	Open	List	Open
	acc.	Dup.	Order	Dup.	Dup.
<i>One canvas holds all n calls</i>					
Dream, $k=16$	81.7	15.0	11.7	100.0	17.3
Dream, $\tau=0.9$	86.5	0.0	48.3	0.0	–
Dream, $k=1$	88.5	0.0	48.3	0.0	0.0
LLaDA2.0, $k=16$	81.7	0.0	96.7	4.4	–
LLaDA2.0, $k=1$	88.5	0.0	100.0	0.0	–
Qwen2.5 (AR)	91.3	0.0	81.7	0.0	–
<i>A team of n Qwen2.5 agents, one call each</i>					
same time, anonymous	0.0	100.0	0.0	100.0	99.7
same time, numbered	0.0	100.0	0.0	97.8	100.0
same time, told the rule	34.6	70.0	8.3	–	–
turns, anonymous	88.5	0.0	91.7	0.0	–
turns, numbered	88.5	0.0	91.7	0.0	–
<i>A team of n Dream agents, one call each</i>					
same time, anonymous	0.0	100.0	0.0	100.0	87.7
same time, numbered	28.8	78.3	3.3	100.0	75.0
same time, told the rule	48.1	45.0	10.0	–	–
turns, anonymous	84.6	0.0	35.0	0.0	–
turns, numbered	85.6	0.0	26.7	0.0	–

5 Symmetric Requests: Simultaneous Deciders Collide

We now take the order away. A *choose- N* request asks for the weather in $n \in \{2, 3, 4\}$ cities without saying which call takes which city, either from a list (“...any three of these cities: Boston, Dallas, Toronto, Austin, Chicago and Paris”, $n + 3$ cities; 60 requests) or openly (“...three different US cities of your choice”; 45 requests). Any n different allowed cities are correct, so the deciders must spread over the candidates, as in a dispersion game [8]. Every city slot holds one token, so each call makes a single decision; in open requests this token also limits which cities can be named, so there we report only duplicates. At 16 tokens per step, every city slot moves in the first step from the all-masked canvas. Each slot is then an isolated decider that sees the whole skeleton but none of the other choices: the information a team of agents would have if every agent saw the full plan.

One canvas. Same-step slots collide (Table 2, right). Dream at 16 tokens per step duplicates a city in every open request and in 15.0% of list requests; pair by pair, 70.0% of the open pairs and 5.5% of the list pairs take the same city. With one token per step, or left to

right, no request has a duplicate, and neither with the confidence threshold: two city slots move together only in 8 list requests, where each is confident about a different city; in every other step at most one slot reaches 0.9, so the slots move one at a time. LLaDA2.0’s calls take turns block by block and rarely collide (at most 4.4% of open requests; 2 of its 76 same-block, same-step pairs). The AR agent never collides.

Teams of agents. Anonymous agents that act at the same time receive identical inputs and make identical calls in every team, a sanity check. Agents that take turns never collide. Agents that act at the same time collide whether or not they are numbered: numbered Qwen2.5 agents take the same city in every list team, and numbered Dream agents in 78.3% (numbering lowers the duplicates by 21.7 points, [11.7, 31.7]). Told the convention, Qwen2.5 and Dream agents still collide in 70.0% and 45.0% of list teams, and take the cities in list order in only 8.3% and 10.0%. What the agents share is primary salience, the choice each finds salient for itself, rather than a focal point chosen to match the others [21]: every anonymous or numbered Qwen2.5 agent that acts at the same time takes the city listed first (99.0–99.8% at $T = 0.7$), and 55.0% of those told the rule do. Sampling does not break the symmetry. Independent uniform choices among the listed cities would collide in 43.2% of list requests (20.0%, 44.4%, and 65.0% for $n = 2, 3, 4$). At $T = 0.7$, with a separate seed for every agent, teams that act at the same time collide in 75.0–100% of list runs, while Dream’s same-step slots collide in 17.3%. LLMs tend to choose alike [3]: what pure coordination rewards, a dispersion game punishes.

What carries over. The protocol carries over from slots to agents. Deciders that move at the same time collide when nothing orders the choices (in 97.8–100% of open requests, for Dream’s same-step slots and the greedy teams), and deciders that take turns do not. The role carries over only when it is a position on the shared skeleton. Dream’s same-step slots differ in 94.5% of list pairs although they see the same canvas, so their positions change the shared policy’s output, though not by list order (11.7% of requests). A number in an agent’s prompt, or a stated rule, leaves most simultaneous agents colliding. P2 holds for the protocol, and the numbered agents fail it as they fail P1.

6 Slot Length Is a Signal That Single Deciders Obey

We now turn to the interface. Sections 4 and 5 gave every slot exactly as many masks as its reference value has tokens. An agent in deployment cannot do this, because it does not know the values before it decodes them.

One extra mask. With one token per step, one extra mask per slot lowers Dream’s set accuracy from 89.8% to 15.5% and LLaDA2.0’s from 92.0% to 16.0% (on its 100 requests; Figure 2a,b). The share of Dream’s slots that hold their own value falls from 96.0% to 37.5%, and 44.4% hold their own value followed by more (Table 3). In Figure 1, HSBC becomes “HSBC Bank” and the loan of \$500,000 one of \$5,000,000; four extra masks make them “HSBC Holdings, Plc” and \$5,000,000,000. On the same scale, committing 16 tokens per step instead of one costs 5.3 points of set accuracy, and one extra mask costs 74.3. Near misses hurt most: set accuracy is lowest with

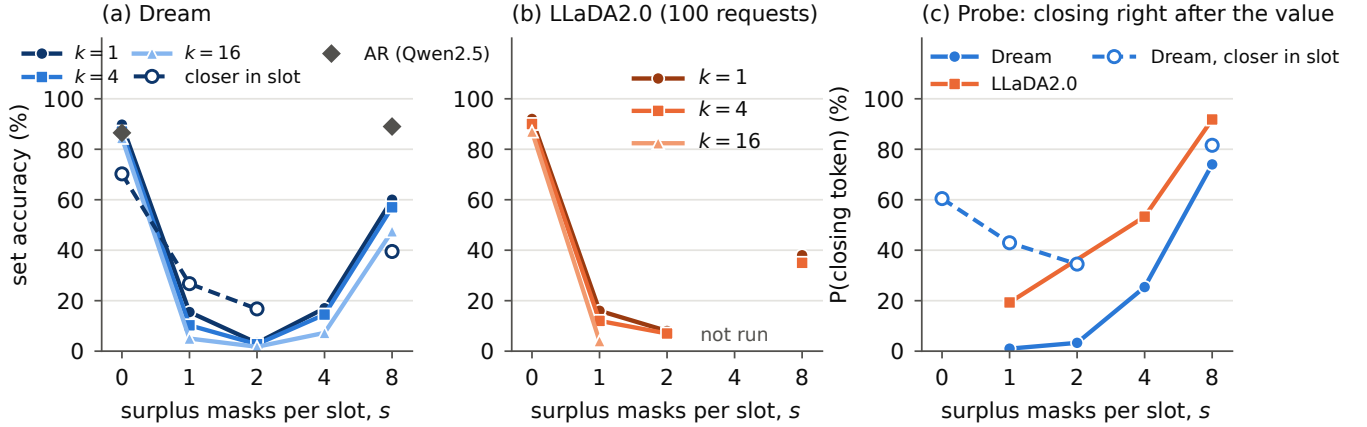


Figure 2: Every slot gets s masks more than its reference value has tokens. (a, b) Set accuracy against the surplus s , one line per number of tokens committed per step; LLaDA2.0 on its 100-request subset, where $s = 4$ was not run and $k = 16$ only up to $s = 1$. Dashed: Dream with the closing token in the slot (Section 6). The AR agent stops at its closing token, so surplus masks only lift its cut-off. (c) Teacher-forced probe over all 3,063 slots: with the reference value written into the slot and s masks left before the skeleton’s closing delimiter, the mean probability that the next token closes the value. Dashed: with the closing token in the slot, where $1 + s$ positions remain.

Table 3: Slot length as a signal (Dream, one token per step). Share of slots (%) that hold their own value, their own value followed by more (overflow), the value of a sibling whose length fits the slot, or anything else: over all 3063 slots (exact, +1), the 514 slots whose length the swap changes, and the 243 slots lengthened to a sibling’s length while the sibling keeps its exact slot (one slot longer). Each slot’s value is read from its own tokens. Set accuracy (%) on the 400 requests, or on the 165 whose slots the swap changes. Agents are anonymous; overflow is counted under other.

	Own	Over- fill	Sibling that fits	Other	Set acc.
<i>One canvas, closing token after the slot</i>					
exact	96.0	0.2	0.9	3.0	89.8
+1	37.5	44.4	0.2	17.9	15.5
swap	5.4	2.3	79.4	12.8	67.3
one slot longer	21.8	20.2	35.4	22.6	17.6
<i>One canvas, closing token in the slot</i>					
exact	89.9	4.8	0.8	4.5	70.3
+1	62.5	26.2	0.1	11.2	26.8
swap	24.7	9.3	49.2	16.7	35.8
one slot longer	40.3	23.9	14.0	21.8	24.8
<i>One Dream agent per call, swapped slots</i>					
same time, exact	85.8	–	4.9	9.3	48.5
same time, swap	1.9	–	89.1	8.9	46.7
turns, swap	2.3	–	84.2	13.4	75.8

two extra masks (3.3%) and recovers with eight (60.0%; LLaDA2.0: 38.0%). The probe in Figure 2c shows why: with one or two masks left after the value, Dream gives a closing token only 1.0% and 3.3% of its probability, and with eight, 74.0%.

The closing delimiter. In this interface, ending a value early means writing a closing token right before the one the skeleton has already written, a pattern the models have not seen in training. To separate this from reading the mask count, we move the closing token into the slot: the skeleton no longer writes it, the slot gets one more position, and after the model’s closing token the slot is filled with spaces, which leaves ordinary JSON (Table 3, middle; dashed in Figure 2). The delimiter accounts for about half of the effect on the slots. With one extra mask, the share of slots that hold their own value falls from 89.9% to 62.5% instead of from 96.0% to 37.5%, and the probability of closing with one extra mask rises from 1.0% to 43.0%. The other half remains: 26.2% of the slots still overflow, set accuracy is still lowest with two extra masks (70.3%, 26.8%, 16.8%, and 39.5% for $s = 0, 1, 2, 8$), and numbers still rarely close early (integers with 19.3% at one extra mask, against 80.8% when only the closing position is left), so an extra digit still multiplies an amount by ten. S³ [33] reports the same over-generation for schema fields with ample masks. The variant is no remedy as it stands: Dream often fails to close a value or closes it with a merged token such as `}}`, so 7.6% of its slots never close, and set accuracy with exact lengths drops to 70.3%.

Swapped lengths and one longer slot. Rotating the slot lengths within each sibling group gives every changed slot the length of a sibling’s value. With one token per step, 79.4% of Dream’s swapped slots then hold the sibling value that fits, both when the slot becomes longer than its own value, which would fit and could close early (79.1%), and when it becomes shorter (79.7%). With the closing token in the slot, 49.2% still do. The swap could also act through taking turns: a slot made too short cannot hold its own value, and a longer slot that moves after it may avoid the value it took. When only one slot of a group is lengthened and its sibling keeps an exact slot, the longer slot takes the sibling’s value in 35.4% of cases, more often than its own (21.8%), while the sibling keeps its own value

Table 4: What an evaluator sees. Requests (%) with each kind of cross-call error, when the agent commits 16 tokens per step with exact slot lengths (shaded) and when it commits one token per step with wrong slot lengths (for estimated lengths also 16), on the same requests. Estimate: lengths predicted by the model in one forward pass (Section 8). Dup.: a duplicated call; bound: a value that belongs to another call; chim.: a string spliced from two calls’ values; shared: an argument that the reference shares across calls and the prediction does not.

Slot lengths	<i>n</i>	Dup.	Bound	Chim.	Shared	CCER
<i>Dream</i>						
exact, $k=16$	400	0.5	0.8	0.0	0.8	2.0
estimate, $k=1$	400	1.5	3.5	1.0	2.3	7.5
estimate, $k=16$	400	3.0	4.3	1.8	3.8	11.0
+1, $k=1$	400	0.5	0.8	3.0	1.5	5.8
+8, $k=1$	400	1.0	0.5	0.0	0.3	1.8
exact, $k=16$	165	0.0	0.6	0.0	0.6	1.2
swap, $k=1$	165	0.0	15.2	1.8	0.0	16.4
<i>LLaDA2.0</i>						
exact, $k=16$	100	1.0	1.0	0.0	1.0	3.0
estimate, $k=1$	100	0.0	6.0	2.0	7.0	14.0
+1, $k=1$	100	2.0	1.0	7.0	4.0	12.0
+8, $k=1$	100	11.0	26.0	1.0	3.0	36.0
exact, $k=16$	34	2.9	2.9	0.0	2.9	8.8
swap, $k=1$	34	0.0	29.4	17.6	0.0	44.1

in 90.9%. The two calls then share a value, which no avoidance of duplicates explains.

A single agent obeys it too. The signal needs no shared canvas. With exact lengths on the 165 swap requests, where sibling slots differ in length, anonymous Dream agents that act at the same time reach 48.5% set accuracy, against 0.6% for Qwen2.5 agents, whose decoding ignores the mask count (difference 47.9 points, [40.0, 55.8]), and 0% on the team-symmetric requests, where the agents that call the same tool see the same skeleton (Table 2). The slot length assigns their roles. With swapped lengths, 89.1% of the swapped slots of agents that act at the same time take the sibling value that fits, and 84.2% of those of agents that take turns and see the calls already made (Table 3, bottom).

P3 holds. A decider reads its mask count as the length of its value. A closing delimiter written after the slot makes this signal binding, but the signal is there without it, and a single agent obeys it as the slots of one canvas do.

7 Length Errors Masquerade as Coordination Failures

The worry about parallel decoding predicts particular errors: a value bound to the wrong call, a duplicated call, a value spliced from two calls. Wrong slot lengths produce exactly these errors with one token per step, where no two decisions are ever made at once (Table 4).

The length outranks the order. A swap sets two cues against each other: the mention order points each slot to its own value, the length

to a sibling’s. Dream follows the length about 15 times as often as the order (79.4% against 5.4% of swapped slots), and LLaDA2.0 about 4 times as often (47.5% against 12.5%, on 34 requests). With one token per step this is how single deciders weigh their cues, not a failure to coordinate, but its result looks like one: the rotated sibling group follows the length while the other arguments of the call follow the order, so the arguments of one call refer to different entities (Figure 1). Where a whole group rotates together, the turn stays correct as a set: 108 of the 111 swap requests that Dream gets right are right only as a reordered set of calls.

What an evaluator sees. With the lengths that the model estimates itself (Section 8), the condition closest to deployment, the two factors separate cleanly. Dream’s CCER is 1.3% with one token per step and 2.0% with 16 when the lengths are exact, and 7.5% and 11.0% when they are estimated. Estimated lengths add 6.3 and 9.0 points, parallel commits 0.8 and 3.5. One token per step with estimated lengths thus has about four times the cross-call errors of 16 tokens per step with exact lengths (difference 5.5 points, 95% interval [3.0, 8.5]; LLaDA2.0: 14.0% against 3.0%, [4.0, 19.0]). Chimera values, which splice individually likely tokens of two values into one (ParallelBench’s “New City” from “New York” and “Mexico City”, which it attributes to parallel decoding [11]), appear one token per step: in 3.0% of Dream’s requests with one extra mask and 7.0% of LLaDA2.0’s, against none at $k = 16$ with exact lengths. LLaDA2.0 writes “HSBC Fargo” into the first call of Figure 1. With eight extra masks, LLaDA2.0 duplicates a call in 11.0% of its requests and binds a value to the wrong call in 26.0%; Dream with eight extra masks is the exception, with as few cross-call errors as with exact lengths (1.8% against 2.0%).

What the wrong actions would do. Executed, the calls would do harm: with one extra mask and one token per step, 33.3% of Dream’s turns would pass a number off by a factor of ten or more (none with exact lengths); with its own length estimates, 19.8% would.

An evaluation protocol. These results concern skeleton interfaces; we do not claim that the errors reported for other dLLM agents [11, 18] come from slot lengths. But in a skeleton interface, an error that persists with one token per step is not an error of parallel decoding. Before attributing a dLLM agent’s cross-call errors to parallel commits, an evaluator should decode the same requests with one token per step and with controlled slot lengths (exact, swapped, and surplus), and attribute to parallelism only the difference that remains.

8 Designing and Evaluating dLLM Agents

An agent in deployment must give every slot a length before it knows the value. We tested two ways to cope with this (Table 5).

A bias toward closing. Adding β to the logits of the closing and padding tokens in every slot rescues large surpluses: with eight extra masks per slot, Dream’s set accuracy at $k = 4$ rises from 57.0% to 72.3% at $\beta = 2$, against 87.0% with exact lengths. It does not rescue near misses: with two extra masks, set accuracy stays at 2.8% at $\beta = 2$ and reaches only 13.8% at $\beta = 8$, a bias that empties many slots of the large surplus, where only 37.3% of the output

Table 5: Coping with unknown value lengths. (a) A logit bias β toward the closing and padding tokens of every slot, Dream at $k=4$ with s surplus masks per slot (exact lengths: 87.0% set accuracy). Parses: share of output that parses. (b) Slot lengths that the model predicts in one forward pass: share of slots whose estimate is exact, too long, or too short, and set accuracy when the agent decodes with the estimates, next to exact lengths on the same requests (shaded). All values in %.

(a) Closing bias, Dream, $k=4$					
β	0	2	4	8	
Set acc., $s=2$	2.8	2.8	6.5	13.8	
Set acc., $s=8$	57.0	72.3	71.3	20.3	
Parses, $s=8$	89.0	93.3	88.3	37.3	
(b) One-forward length estimate					
	Slots			Set acc.	
	Exact	Long	Short	$k=1$	4 16
Dream (400)	51.6	44.2	4.1	42.3	39.0 32.3
exact lengths	–	–	–	89.8	87.0 84.5
LLaDA2.0 (100)	72.8	12.6	14.7	39.0	38.0 –
exact lengths	–	–	–	92.0	90.0 –

then parses. We chose β on the evaluation requests, so 72.3% is optimistic.

Lengths from the model. We give every slot as many masks as its type allows, run one forward pass, read the probability of a closing token at each position as the hazard that the value ends there, and take the most likely length. CAL [17], which searches a single span’s length with 11–18 extra forward passes, may be more accurate; we did not run it. Our estimate is exact for 51.6% of Dream’s slots and 72.8% of LLaDA2.0’s, but all of a request’s slots are exact in only 14.5% and 27.0% of the requests. With these lengths and one token per step, Dream reaches 42.3% set accuracy and LLaDA2.0 39.0%, against 89.8% and 92.0% with exact lengths. Dream’s wrong estimates are mostly too long (91.5%, by a median of 7 tokens), where a closing bias helps (not run together); LLaDA2.0’s are as often too short, which no bias repairs.

Recommendations. The interface writes the signals that the deciders read, the protocol decides who moves when, and the evaluation decides what is blamed. (1) *Interface.* Right slot lengths assign roles and wrong ones reassign them, so avoid near misses: a slot one or two tokens too long is the worst choice, and a slot too short cannot be recovered. A closing delimiter written after the slot doubles the share of values that a near miss corrupts; writing it in the slot halves that share but costs accuracy unless the model closes reliably. Generous room with a closing bias reached 72.3% in our tests, with the surplus counted from the true length. (2) *Protocol.* A confidence threshold lets ordered calls move together (89.0% with 1.9 forward passes per request) and makes symmetric calls move one at a time, because two symmetric choices are rarely confident at once (Section 5). Teams of agents should take turns, or let one model fill a shared skeleton in which each call has its position; a number or a rule in the prompt does not keep simultaneous agents apart, and neither does sampling. (3) *Evaluation.* Only the difference

between parallel commits and one token per step, under the same controlled lengths, is a cost of parallelism (Section 7).

9 Discussion, Limitations, and Conclusion

Beyond one agent. Nothing in the model of Section 3 depends on diffusion: parallel deciders that cannot see each other coordinate when a shared order assigns roles to positions in a shared structure, collide when roles are symmetric unless they take turns, and obey unintended signals of the structure, such as slot lengths. Multi-agent LLM systems in which workers claim subtasks of a shared plan at the same time [19, 27] face the same cases. Our teams suggest that such systems cannot rely on role names or rules in the prompt, or on sampling, to keep simultaneous workers apart, and that turn-taking, or a shared structure that gives each worker its place, is safer. We tested 7B-scale models, one prompt wording per protocol, and single-turn requests, so for larger systems this is a hypothesis, not a finding.

Limitations. The oracle skeleton fixes the number of calls, their tools, and their argument names, so our numbers isolate the values and do not describe a deployed agent: without the skeleton, Dream reaches 75.3% set accuracy at $k = 2$, against 88.0% with it. Exact lengths come from the reference values tokenized on their own, which can be shorter than in context: lifting the AR agent’s cut-off with eight extra masks changes its output in 44 of 400 requests and raises its set accuracy from 86.5% to 89.0%. We tested two dLLMs, which differ in architecture, size, and tokenizer, with greedy decoding except on choose-N; LLaDA2.0 ran on subsets of 100 requests (surplus and estimated lengths) and 34 (swap), where a rate near 40% has a 95% interval of about ± 10 points (39.0% with estimated lengths: [30.0, 48.8]) and, on 34 requests, ± 16 , and not in its native tool-calling format. Our agents saw a skeleton of their own call only, with one wording per protocol; on BFCL they lacked the view of the whole skeleton that the slots had. The open choose-N variant gives each city one token. We did not run a prompted placeholder baseline in the style of S³ [33], the length search of CAL [17], the closing bias with estimated lengths, or a variable-length dLLM such as DreamOn [32].

Conclusion. Parallel deciders that cannot see each other coordinate on what their shared structure gives them. The same-step slots of one dLLM agent follow the request’s order through their positions, while LLM agents that act at the same time cannot use a number, or even a rule, in their prompts; taking turns works for both. The structure also sends signals that no one intended: a dLLM decider reads its slot length as the length of its value, so wrong lengths bind values to the wrong calls even one token per step, and the errors look like failures of parallel decoding. Designers should give parallel deciders their roles through structure and get slot lengths right; evaluators should control slot lengths before they blame parallelism.

Use of AI Tools

We used Claude Opus 5.5 (Anthropic) in this work.

References

- [1] Saaket Agashe, Yue Fan, Anthony Reyna, and Xin Eric Wang. 2025. LLM-Coordination: Evaluating and Analyzing Multi-agent Coordination Abilities

- in Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 8053–8072. doi:10.18653/v1/2025.findings-naacl.448
- [2] Ido Aharon, Emanuele La Malfa, Michael Wooldridge, and Sarit Kraus. 2026. Tacit Coordination of Large Language Models. *arXiv preprint arXiv:2601.22184* (2026).
 - [3] Gonzalo Ballesterio, Hadi Hosseini, Samarth Khanna, and Ran I. Shorror. 2026. Strategic Algorithmic Monoculture: Experimental Evidence from Coordination Games. *arXiv preprint arXiv:2604.09502* (2026).
 - [4] Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. 2025. Accelerated Sampling from Masked Diffusion Models via Entropy Bounded Unmasking. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. doi:10.52202/085713-1874
 - [5] Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, et al. 2025. LLaDA2.0: Scaling Up Diffusion Language Models to 100B. *arXiv preprint arXiv:2512.15745* (2025).
 - [6] Meihua Dang and Stefano Ermon. 2026. Constrained Decoding for Diffusion Language Models via Efficient Inference over Finite Automata. In *Advances in Neural Information Processing Systems (NeurIPS 2026)*. To appear. <https://openreview.net/forum?id=ssAyzXEQoT>
 - [7] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 10932–10952. doi:10.18653/v1/2023.emnlp-main.674
 - [8] Trond Grenager, Rob Powers, and Yoav Shoham. 2002. Dispersion Games: General Definitions and Some Specific Learning Results. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2002)*. 398–403. doi:10.5555/777092.777156
 - [9] Boyan Han, Yiwei Wang, Yi Song, Yujun Cai, and Chi Zhang. 2026. Dynamic Infilling Anchors for Format-Constrained Generation in Diffusion Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 26213–26227. doi:10.18653/v1/2026.acl-long.1205
 - [10] Hengyuan Hu, Adam Lerer, Alex Peysakhovich, and Jakob Foerster. 2020. “Other-Play” for Zero-Shot Coordination. In *Proceedings of the 37th International Conference on Machine Learning (ICML 2020) (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 4399–4410. <https://proceedings.mlr.press/v119/hu20a.html>
 - [11] Wonjun Kang, Kevin Galim, Seunghyuk Oh, Minjae Lee, Yuchen Zeng, Shuibai Zhang, Coleman Hooper, Yuezhou Hu, et al. 2026. ParallelBench: Understanding the Trade-offs of Parallel Decoding in Diffusion LLMs. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=OsZr5T7Cd0>
 - [12] Aja Khanal, Kaushik T. Ranade, Rishabh Agrawal, Kalyan S. Basu, and Apurva Narayan. 2026. Agents of Diffusion: Enhancing Diffusion Language Models with Multi-Agent Reinforcement Learning for Structured Data Generation. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026)*. International Foundation for Autonomous Agents and Multiagent Systems, 449–458. doi:10.65109/GGJL7344
 - [13] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 24370–24391. <https://proceedings.mlr.press/v235/kim24y.html>
 - [14] David K. Lewis. 1969. *Convention: A Philosophical Study*. Harvard University Press, Cambridge, MA.
 - [15] Jinsong Li, Xiaoyi Dong, Yuhang Zang, Yuhang Cao, Jiaqi Wang, and Dahua Lin. 2026. Beyond Fixed: Training-Free Variable-Length Denoising for Diffusion Large Language Models. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=lc2A2gCscC>
 - [16] Anji Liu, Oliver Broadrick, Mathias Niepert, and Guy Van den Broeck. 2025. Discrete Copula Diffusion. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. 88953–88979.
 - [17] Hengchang Liu, Zhao Yang, and Bing Su. 2026. Diffusion LMs Can Approximate Optimal Infilling Lengths Implicitly. *arXiv preprint arXiv:2602.00476* (2026).
 - [18] Qingyu Lu, Liang Ding, Kanjian Zhang, Jinxia Zhang, and Dacheng Tao. 2026. The Bitter Lesson of Diffusion Language Models for Agentic Workflows: A Comprehensive Reality Check. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 43997–44020. doi:10.18653/v1/2026.acl-long.2036
 - [19] Yuzhen Mao, Jerry Gu, Aadi Chauhan, Qizheng Zhang, Hangoo Kang, and Azalia Mirhoseini. 2026. Decentralized Multi-Agent Systems with Shared Context. *arXiv preprint arXiv:2606.10662* (2026).
 - [20] Jacob Marschak and Roy Radner. 1972. *Economic Theory of Teams*. Yale University Press, New Haven, CT.
 - [21] Judith Mehta, Chris Starmer, and Robert Sugden. 1994. The Nature of Salience: An Experimental Investigation of Pure Coordination Games. *American Economic Review* 84, 3 (1994), 658–673.
 - [22] Niels Mündler, Jasper Dekoninck, and Martin Vechev. 2026. Constrained Decoding of Diffusion LLMs with Context-Free Grammars. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=7Sph4KyeYO>
 - [23] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large Language Diffusion Models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. doi:10.52202/085713-1689
 - [24] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 48371–48392. <https://proceedings.mlr.press/v267/patil25a.html>
 - [25] Jiacheng Qiu, Christopher E. Mower, Jan Peters, Haitham Bou-Ammar, and Matthieu Zimmer. 2026. Does This Action Still Explain the Task? Reverse Scoring for Diffusion Language Model Agents. *arXiv preprint arXiv:2609.38536* (2026).
 - [26] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
 - [27] Gleb Rodionov, Roman Garipov, Alina Shutova, George Yakushev, Erik Schultheis, Vage Egiazarian, Anton Sinitin, Denis Kuznetsov, and Dan Alistarh. 2025. Hoggwild! Inference: Parallel LLM Generation via Concurrent Attention. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. doi:10.52202/085713-1551
 - [28] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. Simple and Effective Masked Diffusion Language Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. doi:10.52202/079017-4135
 - [29] Thomas C. Schelling. 1960. *The Strategy of Conflict*. Harvard University Press, Cambridge, MA.
 - [30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. doi:10.52202/075280-2997
 - [31] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. 2026. Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=3Z3s6hnOT>
 - [32] Zirui Wu, Lin Zheng, Zhihui Xie, Jiacheng Ye, Jiahui Gao, Shansan Gong, Yansong Feng, Zhenguo Li, et al. 2026. DreamOn: Diffusion Language Models For Code Infilling Beyond Fixed-size Canvas. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=EQTpmqukiU>
 - [33] Zhen Xiong, Yujun Cai, Zhecheng Li, and Yiwei Wang. 2026. Unveiling the Potential of Diffusion Large Language Model in Controllable Generation. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=qhd0qv6L0k>
 - [34] Jingyi Yang, Yuxian Jiang, and Jing Shao. 2026. ρ -EOS: Training-free Bidirectional Variable-Length Control for Masked Diffusion LLMs. *arXiv preprint arXiv:2601.22527* (2026).
 - [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*. https://openreview.net/forum?id=WE_vluYUL-X
 - [36] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7B: Diffusion Large Language Models. *arXiv preprint arXiv:2508.15487* (2025).
 - [37] Jiahao Zhao, Shaoxuan Xu, Zhongxiang Sun, Fengqi Zhu, Jingyang Ou, Yuling Shi, Chongxuan Li, Xiao Zhang, and Jun Xu. 2026. DLLM-Searcher: Adapting Diffusion Large Language Model for Search Agents. *arXiv preprint arXiv:2602.07035* (2026).
 - [38] Huiling Zhen, Weizhe Lin, Renxi Liu, Kai Han, Yiming Li, Yuchuan Tian, Hanting Chen, Xiaoguang Li, et al. 2026. DLLM Agent: See Farther, Run Faster. *arXiv preprint arXiv:2602.07451* (2026).